

Communication Systems Project Report

Receiver Design

Afolabi Afolayan 27813
Dana Darafeja 27669

WS 25/26

Project Architecture

For our project, we adopt the receiver architecture presented in the SRD textbook, Chapter 15.2, “*A Design Methodology for the M6 Receiver.*” (Rice, 2011). This methodology decomposes the receiver into a sequence of functional stages, each responsible for mitigating a specific class of channel impairments.

The following sections describe selected stages of our receiver implementation, focusing on their goals, operating principles, and implementation logic.

Stage 0 : Notch filter

For especially distorted signals (like our mystery signal C), there is noise in the baseband. In these cases, we identify the noise frequency components and use a notch filter to target them specifically for attenuation, thereby improving our SNR.

Note: that is only used on mystery signal C in our code.

The Octave code for the filter:

```
r_notch = r;
for f0 = [90e3, 100e3]
    w0 = 2*pi*f0/f_s;
    rr = 0.998;
    b = [1, -2*cos(w0), 1];
    a = [1, -2*rr*cos(w0), rr^2];

    r_notch = filter(b, a, r_notch);
end
r = r_notch;
end
```

Stage 1 : Carrier Recovery and Demodulation

Our received signal can have several “real world” impairments, for example, its frequency component could be moved higher or lower on the frequency axis (frequency offset), or the signal could also be rotated around the axis (phase offset).

These impairments cause the constellation points to deviate from their ideal locations, which can lead to symbol decision errors and loss of information. To correct these effects, a phase-locked loop (PLL) is applied at the receiver.

In our system, carrier recovery is implemented using a Costas loop, which simultaneously corrects carrier phase and small frequency offsets. We use this information to demodulate our signal to baseband with the help of a carrier and use a low pass filter to remove high frequency components.

Operating Principle of the Costas Loop

The Costas loop operates by iteratively rotating the received signal until its imaginary component is minimized. The Octave code:

```
bw = (1+srrc_rolloff)/2 * Rs;
wp = min(2*bw/f_s, 0.45);
ws = min(1.25*wp, 0.50);
fl = 100;
ff = [0, wp, ws, 1];
fa = [1, 1, 0, 0];
h = firpm(fl, ff, fa);

mu = 0.02;
theta = zeros(N,1);
zs = zeros(fl+1,1);
zc = zeros(fl+1,1);
w0 = 2*pi*fc;

for k = 1:N-1
    ph = w0*t(k) + theta(k);
    zs = [zs(2:fl+1); 2*r(k)*sin(ph)];
    zc = [zc(2:fl+1); 2*r(k)*cos(ph)];
    lpfs = flipud(h(:))' * zs;
    lpfc = flipud(h(:))' * zc;
    theta(k+1) = theta(k) - mu*lpfs*lpfc;
end

c = cos(2*pi*fc*t + theta');
r_dm = r.*c ;

r_bb = 2*filter(h,1,r_dm);
```

The algorithm works as follows:

1. Design a low pass filter to get the baseband (determined by symbol rate and rolloff)
2. Set the step size
3. In the loop:
 - A carrier frequency estimate and a low pass filter is used to mix the received signal down to baseband.
 - The baseband signal is separated into in-phase (lpfs) and quadrature (lpfc) components.
 - An error signal is computed from the interaction between the in-phase and quadrature components.
 - This error signal is fed back to update the phase estimate.
 - The loop continuously adjusts the phase until the quadrature component approaches zero.
4. After getting the phase offset we create our carrier signal, modulate with our received signal to demodulate, and filter it to get the baseband

After this stage we get a phase-aligned baseband signal in which the quadrature component approaches zero and the symbols lie along the real axis.

The loop may lock with a 180° ambiguity, additional frame synchronization is done later on.

Stage 2 : Matched filter (SRRC)

To maximize SNR of the received signal we use a matched filter to correlate the pulse shape with the received signal. The Octave code:

```
pulse = srrc(srrc_length, srrc_rolloff, round(M));
pulse = pulse/sqrt(sum(pulse.^2));
r_mf = filter(fliplr(pulse), 1, r_bb);
```

Once the SRRC pulse is squared (which is convolution in time domain) with the same shape as it was pulse shaped with, this creates a raised cosine peak at the desired symbols times and ripples crossing the x axis only at $y=0$, which we can then use to uniquely identify each symbol. That reduces intersymbol-interference.

Stage 3: Timing Recovery (DDTR)

After we have applied a matched filter to the signal, the next stage is to choose an optimal time for the receiver so that its clock is synchronized with that of the transmitter and intersymbol interference is minimized and symbol decisions are most reliable.

We implement a Decision-Directed Timing Recovery (DDTR) algorithm because it operates without requiring a known training sequence, which works for all mystery signals, not only a specific case, and continues to adapt during steady-state operation.

Algorithmic Principle and Octave code

```
l = srrc_length * round(M);
tnow = l + 1;
tau = 0;

mu_tau = 0.01;
delta = 0.5;

r_t = zeros(1, length(r_mf));
tausave = zeros(1, length(r_mf));
i = 0;

while tnow < length(r_mf) - 2*l
    i = i + 1;

    r_t(i) = intersinc(r_mf, tnow + tau, l);
    xdp = intersinc(r_mf, tnow + tau + delta, l);
    xdm = intersinc(r_mf, tnow + tau - delta, l);
    dx = xdp - xdm;

    qx = quantalph(r_t(i), [-3 -1 1 3]);
    tau = tau + mu_tau * dx * (qx - r_t(i));

    tnow = tnow + M;
    tausave(i) = tau;
end
```

In our code the decision-directed timing recovery is implemented as an adaptive feedback loop. The loop iteratively adjusts the sampling offset to minimize decision errors.

The algorithm works as follows:

1. We set out values
2. In the loop:
 - Increment the symbol counter
 - Interpolate the match filter output at the current timing
 - Estimate the slope ie the time derivative (rising, falling or flat)
 - Quantize and calculate the DD error
 - Update the time offset
3. Get our time recovered signal `r_t`

Timing recovery adjusts the sampling instant left or right until the quantizer output becomes maximally confident. In the result, the algorithm slides the sampling point toward the center of the eye diagram.

The recovered signal can then be passed to the frame synchronization.

Stage 4 : Frame Synchronization

The main aim of this stage is to correlate our quantized values with the known preamble PAM symbols. This way we can determine the start of the message that we are trying to decode. Without it irrelevant data would be decoded. Framing defines a window that we put over the code and in the bounds of this window are meaningful symbols that can be decoded. The Octave code:

```
payload_length = userDataLength*4;
frame_length = preamble_length + payload_length;

corr_vals = zeros(length(r_t)-preamble_length+1,1);
corr_sign = zeros(length(r_t)-preamble_length+1,1);

for idx = 1:length(corr_vals)
    seg = r_t(idx:idx+preamble_length-1);
    c = sum(seg(:) .* preamble_syms);
    corr_vals(idx) = abs(c);
    corr_sign(idx) = sign(c + 1e-12);
end

[~, pre_start] = max(corr_vals);
all_syms = [];

k = pre_start;
while k + frame_length - 1 <= length(r_t)
    frame_preamble = r_t(k:k+preamble_length-1);
    frame_corr = sum(frame_preamble(:) .* preamble_syms);

    if frame_corr < 0
        frame_multiplier = -1;
```

```

else
    frame_multiplier = 1;
end

payload_start = k + preamble_length;
payload_end = payload_start + payload_length - 1;

payload_soft = frame_multiplier * r_t(payload_start:payload_end);
payload = quantalph(payload_soft, [-3 -1 1 3]);

all_syms = [all_syms; payload(:)];
k = k + frame_length;
end

n = length(all_syms);
n_valid = floor(n/4)*4;
all_syms = all_syms(1:n_valid);

```

The algorithm works as follows:

1. We take our time recovered signal r_t and slide the preamble across it
2. In the loop:
 - for each frame we check if the preamble is negatively correlated, flip it if it is
 - make classification decision on the values (3,1,-1,-3)
 - collect the symbols

After this stage we are left with all the symbols in the right direction and rounded up/down to their best fitting PAM level.

Stage 5 : Decode

This is the final stage where we take our frame synchronized symbols and convert it back to human readable characters through the ASCII code which is a predefined agreed upon mapping list of PAM to characters.

Contributions of project members

This project was completed through a collaborative effort between Afolabi Afolayan and Dana Darafeja. While both contributed to all aspects, Afolabi Afolayan primarily focused on design and implementation of notch filter, demodulation and carrier recovery, frame synchronization and report, and Dana Darafeja concentrated on design and implementation of matched filter, time recovery, frame synchronization and report.

References

M. Rice, "Software Receiver Design," 2011

Notes

A.I. (ChatGPT and Claude) were used to assist in the creation of mental models and debugging code.